

**PROTOTIPO WEB PARA APOYAR LA ENSEÑANZA DE FUNDAMENTOS DE
INTERPRETACIÓN Y COMPILACIÓN DE LENGUAJES DE PROGRAMACIÓN
MEDIANTE LENGUAJES ORIENTADOS POR SINTAXIS**

JOTA LÓPEZ RAMÍREZ

Código: 2259394

jota.lopez@correounivalle.edu.co

ESMERALDA RIVAS GUZMÁN

Código: 2259580

esmeralda.rivas@correounivalle.edu.co

**FACULTAD DE INGENIERÍA
ESCUELA DE INGENIERÍA DE SISTEMAS Y COMPUTACIÓN
PROGRAMA ACADÉMICO DE INGENIERÍA DE SISTEMAS
TULUÁ – VALLE DEL CAUCA**

2026

**PROTOTIPO WEB PARA APOYAR LA ENSEÑANZA DE FUNDAMENTOS DE
INTERPRETACIÓN Y COMPILACIÓN DE LENGUAJES DE PROGRAMACIÓN
MEDIANTE LENGUAJES ORIENTADOS POR SINTAXIS**

JOTA LÓPEZ RAMÍREZ

Código: 2259394

jota.lopez@correounivalle.edu.co

ESMERALDA RIVAS GUZMÁN

Código: 2259580

esmeralda.rivas@correounivalle.edu.co

Director

Carlos Andrés Delgado Saavedra, Ing.

carlos.andres.delgado@correounivalle.edu.co

FACULTAD DE INGENIERÍA

ESCUELA DE INGENIERÍA DE SISTEMAS Y COMPUTACIÓN

PROGRAMA ACADÉMICO DE INGENIERÍA DE SISTEMAS

TULUÁ – VALLE DEL CAUCA

2026

Tabla de Contenido

Resumen	v
Introducción	vi
1. Contexto y objetivos	1
1.1. Problema de Investigación	1
1.1.1. Descripción del problema	1
1.1.2. Formulación del problema	3
1.2. Objetivos	3
1.2.1. Objetivo general	3
1.2.2. Objetivos específicos y resultados esperados	3
1.3. Alcance	3
1.4. Estructura del documento	4
2. Marco Referencial	5
2.1. Glosario	5
3. Diagnóstico de dificultades conceptuales	6
3.1. Metodología	6
3.1.1. Diseño de los instrumentos	6
3.1.2. Selección de la población	7
3.2. Análisis de la encuesta a estudiantes	7
3.2.1. Resultados cuantitativos	7
3.2.2. Resultados cualitativos	9
3.3. Análisis de la encuesta a docentes	9
3.3.1. Percepción general de los estudiantes	10
3.3.2. Dificultades con paradigmas avanzados	11
3.4. Revisión del microcurrículo	11
3.5. Selección de conceptos base para el prototipo	12
4. Diseño del prototipo FLP Viewer	14
4.1. Especificación de requisitos del sistema	14
4.1.1. Enfoque metodológico	14
4.1.2. Requisitos funcionales	15
4.1.3. Requisitos no funcionales	15

4.2. Diseño del prototipo	16
4.2.1. Arquitectura del sistema	16
4.2.1.1. Infraestructura de despliegue	17
4.2.1.2. Capa de servidor	17
4.2.1.3. Capa de cliente	17
4.2.1.4. Pipeline de gramática	18
4.2.2. Selección de tecnologías	18
4.2.3. Prototipos de interfaz de usuario	19
4.2.4. Estrategias de visualización	22
Bibliografía	24
A. Microcurrículo de FLP	26
B. Encuesta a estudiantes	27
C. Encuesta a docentes	28
D. Requisitos del sistema	29
D.1. Requisitos Funcionales	29
D.2. Requisitos No Funcionales	33

Lista de Figuras

1.1. Resultados de la encuesta aplicada a estudiantes de FLP (n=36)	2
4.1. Diagrama de arquitectura del sistema	18
4.2. Wireframe de la interfaz principal del prototipo FLP Viewer	21
4.3. Wireframe del modal de definición de gramática BNF	22

Lista de tablas

1.1. Objetivos Específicos	3
3.1. Distribución de respuestas - encuesta a estudiantes de FLP ($n = 36$)	8
3.2. Percepción general de los docentes sobre habilidades de los estudiantes ($n = 5$)	10
3.3. Percepción de los docentes de semestres avanzados sobre programación lógica y paradigmas avanzados ($n = 3$)	11
3.4. Indicadores de logro seleccionados como base para el prototipo	13
4.1. Resumen de requisitos funcionales del sistema	15
4.2. Resumen de requisitos no funcionales del sistema	16
4.3. Tecnologías seleccionadas y justificación	19
D.1. RF-01: Generación del AST	29
D.2. RF-02: Visualización gráfica e interactiva del AST	30
D.3. RF-03: Ejecución y evaluación del programa	30
D.4. RF-04: Representación del ambiente de ejecución	31
D.5. RF-05: Biblioteca de ejemplos	31
D.6. RF-06: Soporte y validación de sintaxis en Racket	32
D.7. RF-07: Visualización de los componentes internos del intérprete	32
D.8. RF-08: Plantilla base editable y exportación del entorno	33
D.9. RNF-01: Usabilidad	34
D.10.RNF-02: Rendimiento	34
D.11.RNF-03: Accesibilidad web	35
D.12.RNF-04: Mantenibilidad	35
D.13.RNF-05: Escalabilidad	36

Resumen

// TODO: Escribir el resumen del proyecto, incluyendo los objetivos, metodología y resultados esperados.

Introducción

// TODO: Introducir la problemática y el contexto del estudio.

Capítulo 1

Contexto y objetivos

1.1. Problema de Investigación

1.1.1. Descripción del problema

La interpretación y compilación de lenguajes de programación constituye uno de los ejes centrales en la formación de ingenieros de sistemas. Comprender cómo un programa escrito en un lenguaje de alto nivel es procesado, analizado y ejecutado por un sistema computacional permite sustentar el trabajo en múltiples áreas de la disciplina, desde el desarrollo de compiladores y herramientas de análisis estático, hasta el diseño de lenguajes de dominio específico. Sin embargo, la enseñanza de estos contenidos enfrenta dificultades por sus conceptos abstractos, densos y difíciles de conectar con la práctica [1] [2].

En el programa de Ingeniería de Sistemas de la Universidad del Valle sede Tuluá, la asignatura *Fundamentos de Interpretación y Compilación de Lenguajes de Programación* (FLP) demanda una comprensión teórica sólida y una capacidad de visualización de procesos internos que no resultan evidentes a partir del código fuente [3]. A lo largo de su desarrollo académico, los estudiantes deben asimilar conceptos como el análisis léxico y sintáctico, la construcción y recorrido de árboles de sintaxis abstracta (AST), la representación de ambientes de ejecución, la gestión del alcance léxico y el modelado de clausuras.

La principal herramienta de trabajo de la asignatura es el lenguaje Racket junto con los materiales del libro *Essentials of Programming Languages* (EOPL) [4]. Si bien esta combinación ofrece una base conceptual rigurosa y permite implementar intérpretes funcionales, su aprovechamiento exige una familiaridad previa con el entorno, así como un dominio sólido del paradigma funcional, en el que los estudiantes pueden presentar ciertos vacíos conceptuales. En particular, la sintaxis de prefijo de Racket, su estructura basada en paréntesis y los identificadores propios del entorno tienden a desplazar la atención cognitiva del estudiante desde los conceptos fundamentales de interpretación hacia la resolución de errores sintácticos del lenguaje utilizado [5].

Además de la carga impuesta por el entorno de programación, la asignatura carece de recursos interactivos que permitan a los estudiantes visualizar de manera dinámica y progresiva los procesos internos que se estudian. La literatura en tecnología educativa ha demostrado consistentemente que las representaciones visuales e interactivas favorecen la comprensión de procesos de compilación e

interpretación al hacer tangibles las transformaciones que ocurren en cada etapa del procesamiento de un programa [6] [7]. En ausencia de estos recursos, el aprendizaje depende en gran medida de las explicaciones del docente en clase, lo que reduce la autonomía del estudiante y limita las posibilidades de exploración fuera del aula.

Con el propósito de caracterizar estas dificultades desde la experiencia directa de quienes han cursado la asignatura, se aplicaron dos encuestas: una dirigida a 36 estudiantes que asistieron en distintos períodos académicos, y otra a cinco docentes del programa. Los resultados de ambos instrumentos se analizan en detalle en el Capítulo 3; no obstante, los datos de la encuesta estudiantil permiten fundamentar el planteamiento del problema con evidencia cuantitativa directa.

Como se aprecia en la Figura 1.1, tres hallazgos resultan especialmente significativos. Primero, el 77.8% de los estudiantes reportó haber tenido dificultades para comprender conceptos abstractos como entornos, estado o secuenciación, lo que confirma el carácter problemático de los contenidos de mayor abstracción. Segundo, apenas el 19.4% se siente preparado para aplicar los conceptos aprendidos en otros cursos o proyectos, lo que revela una brecha importante entre la comprensión teórica y la transferencia práctica del conocimiento. Tercero, el 94.5% de los encuestados consideró que recursos adicionales, como laboratorios interactivos, herramientas de visualización o ejemplos prácticos progresivos, habrían mejorado su aprendizaje de manera significativa.

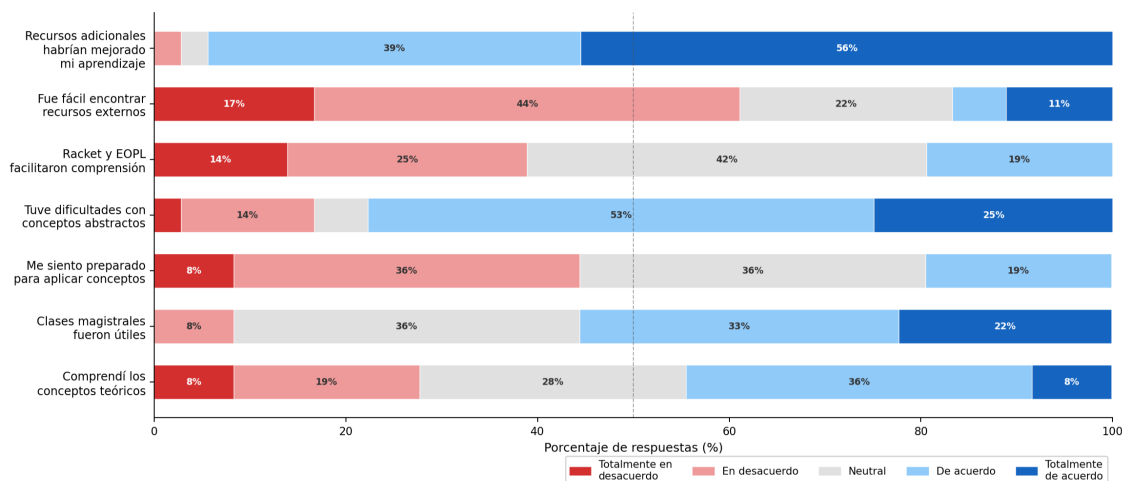


Figura 1.1: Resultados de la encuesta aplicada a estudiantes de FLP (n=36)

Fuente: elaboración propia

En síntesis, las dificultades observadas en la asignatura FLP responden a una unión de factores: el alto nivel de abstracción inherente a los contenidos, la fricción cognitiva que genera el entorno de programación utilizado, la escasez de recursos externos de apoyo, y la ausencia de herramientas interactivas que permitan a los estudiantes explorar y visualizar los procesos que estudian. Esta situación genera dependencia de las clases presenciales, limita la autonomía en el aprendizaje y se traduce en una comprensión superficial de conceptos fundamentales para asignaturas posteriores y para el ejercicio profesional.

1.1.2. Formulación del problema

¿Cómo desarrollar un prototipo web que apoye la comprensión de los contenidos de la asignatura *Fundamentos de Interpretación y Compilación de Lenguajes de Programación* (FLP) de la Universidad del Valle mediante la visualización de estructuras internas y el uso de lenguajes orientados por sintaxis?

1.2. Objetivos

1.2.1. Objetivo general

Desarrollar un prototipo web que apoye la comprensión de los contenidos de la asignatura *Fundamentos de Interpretación y Compilación de Lenguajes de Programación* (FLP) de la Universidad del Valle mediante la visualización de estructuras internas y el uso de lenguajes orientados por sintaxis.

1.2.2. Objetivos específicos y resultados esperados

Objetivo específico	Sección del documento
Identificar las dificultades conceptuales que enfrentan los estudiantes de la asignatura, con el propósito de orientar la selección de ejemplos, casos de uso y mecanismos de visualización.	Capítulo -
Caracterizar los fundamentos teóricos y técnicos relacionados con las gramáticas formales, el análisis sintáctico y la generación de árboles de sintaxis abstracta, que servirán como base conceptual para el diseño del prototipo.	Capítulo -
Diseñar la arquitectura web y las interfaces de usuario, integrando estrategias de visualización que favorezcan la comprensión de las estructuras sintácticas.	Capítulo -
Implementar un prototipo funcional que permita definir gramáticas, visualizar los árboles de sintaxis abstracta y los cambios en el estado durante la evaluación de los programas escritos por el usuario.	Capítulo -
Evaluar la usabilidad y funcionalidad del prototipo mediante pruebas con estudiantes inscritos en la asignatura y con el docente responsable.	Capítulo -

Tabla 1.1: Objetivos Específicos

1.3. Alcance

El presente trabajo está enfocado en el diseño e implementación de un prototipo web educativo orientado a apoyar la comprensión de los conceptos fundamentales de interpretación y compilación de lenguajes de programación en la asignatura FLP del programa de Ingeniería de Sistemas de la Universidad del Valle sede Tuluá.

Desde el punto de vista técnico, el prototipo cubre tres funcionalidades principales: (1) la definición

y edición de gramáticas con sintaxis configurable por el usuario; (2) la visualización de árboles de sintaxis abstracta (AST) generados a partir de dichas gramáticas; y (3) la representación de los cambios en el ambiente durante la evaluación de programas escritos bajo la gramática definida. Estas funcionalidades están orientadas específicamente a los conceptos que, conforme al diagnóstico presentado en el Capítulo 3, presentan mayor dificultad de comprensión en la asignatura.

El alcance del prototipo se limita a funcionalidades demostrativas con fines pedagógicos. No se pretende desarrollar un lenguaje de propósito general ni implementar mecanismos avanzados de ejecución tales como depuración paso a paso, compilación a código máquina, optimización de rendimiento o autenticación y persistencia de usuarios. El sistema tampoco contempla despliegue en infraestructura institucional de uso permanente, puesto que su operación se restringe a un entorno de pruebas durante la fase de evaluación.

La evaluación del prototipo se llevará a cabo con un grupo de entre 10 y 20 estudiantes matriculados en la asignatura durante el período de desarrollo del trabajo, junto con el docente responsable. Dada la naturaleza y el tiempo disponible del proyecto, la evaluación se restringe a aspectos de funcionalidad técnica y usabilidad, mediante la aplicación del cuestionario estandarizado System Usability Scale (SUS) y un cuestionario de utilidad pedagógica de elaboración propia, dejando por fuera evaluaciones de impacto en el aprendizaje a largo plazo, pues estos requieren condiciones experimentales que exceden el alcance de un trabajo de grado.

Finalmente, el prototipo no pretende reemplazar los recursos actuales de la asignatura, en particular lo referente a Racket y EOPL, sino complementarlos, ofreciendo un punto de entrada más accesible y visual a los conceptos que los estudiantes identifican como los de mayor dificultad.

1.4. Estructura del documento

//TODO: Por definir

Capítulo 2

Marco Referencial

2.1. Glosario

Capitulo pendiente

Capítulo 3

Diagnóstico de dificultades conceptuales

Para identificar las dificultades conceptuales que enfrentan los estudiantes de la asignatura FLP, se realizó un diagnóstico basado en tres fuentes complementarias: una encuesta dirigida a estudiantes que han cursado FLP en distintos períodos académicos, una encuesta aplicada a docentes del programa de Ingeniería de Sistemas y una revisión del microcurrículo de la asignatura.

Estas fuentes permitieron obtener una caracterización de las dificultades desde diferentes perspectivas, teniendo la de los estudiantes que las experimentan directamente, la de los docentes que se encargan de enseñar el contenido y la del diseño curricular que estructura el aprendizaje, lo que generó una selección justificada de los conceptos que servirán como base para la biblioteca de casos predefinidos del prototipo.

3.1. Metodología

El diagnóstico se apoyó en un enfoque mixto, combinando métodos cuantitativos provenientes de encuestas con escala Likert y métodos cualitativos a través de preguntas abiertas y análisis documental del microcurrículo.

3.1.1. Diseño de los instrumentos

El diseño de ambas encuestas se fundamentó en las dimensiones de evaluación identificadas en la literatura acerca de las dificultades en la enseñanza de compiladores e intérpretes [3] [6] y sobrecarga cognitiva en programación [8], que orientaron la selección de los ítems a evaluar: comprensión teórica, capacidad de aplicación, percepción de las herramientas, disponibilidad de recursos y valoración de alternativas interactivas. Por otro lado, el microcurrículo oficial de la asignatura permitió anclar las preguntas a los resultados de aprendizaje e indicadores de logro concretos de FLP, evitando que el instrumento midiera percepciones genéricas desvinculadas del contexto específico de la asignatura.

La encuesta a estudiantes fue diseñada con siete ítems de escala Likert de cinco niveles (1: Totalmente en desacuerdo, 5: Totalmente de acuerdo) para evaluar la percepción de las dificultades conceptuales en FLP, junto con una pregunta abierta orientada a identificar las recomendaciones de mejora. Esta

estructura permitió distinguir entre dificultades atribuibles al contenido, al entorno de programación y a la escasez de recursos complementarios.

El diseño de la encuesta a docentes incluyó una pregunta de segmentación, teniendo en cuenta el semestre en el que cada docente dicta programación principalmente, seguida de bloques temáticos con ítems Likert sobre la percepción de las dificultades estudiantiles en distintos paradigmas de programación, cerrando con una pregunta abierta para comentarios o sugerencias respecto al proceso de enseñanza de programación.

Finalmente, la revisión del microcurrículo se realizó sobre el documento oficial de la asignatura FLP (código 750017C) el cual se encuentra en el Anexo A. Este documento estructura los contenidos a través de competencias específicas, resultados de aprendizaje (RA) e indicadores de logro (IL), lo que permitió contrastar las dificultades reportadas por estudiantes y docentes con la estructura formal de los contenidos e identificar cuáles indicadores concentran mayor complejidad conceptual.

3.1.2. Selección de la población

La población objetivo de la encuesta estudiantil fueron estudiantes que hubieran cursado FLP en la Universidad del Valle sede Tuluá durante los últimos tres años académicos, optando por no restringir la convocatoria al período en curso, con el objetivo de ampliar el tamaño de la muestra y capturar perspectivas recientes y en retrospectiva, lo que permitió que el diagnóstico reflejara una experiencia acumulada con la asignatura. La encuesta fue aplicada de forma voluntaria, obteniendo 36 respuestas adjuntas en el Anexo B.

La población objetivo de la encuesta docente fueron los profesores del programa de Ingeniería de Sistemas que dictan asignaturas de programación, independientemente del semestre. Esta decisión busca obtener una perspectiva longitudinal del programa, considerando que mientras los docentes de semestres avanzados informan sobre las dificultades que observan directamente en FLP y asignaturas afines, los docentes de primeros semestres permiten caracterizar las bases con las que los estudiantes ingresan. La encuesta fue aplicada de forma voluntaria, obteniendo 5 respuestas, las cuales se encuentran detalladas en el Anexo C.

3.2. Análisis de la encuesta a estudiantes

3.2.1. Resultados cuantitativos

La Tabla 3.1 presenta la distribución de respuestas para los siete ítems Likert de la encuesta aplicada a los 36 estudiantes.

Tabla 3.1: Distribución de respuestas - encuesta a estudiantes de FLP ($n = 36$)

Ítem	Tot. desac.	En desac.	Neutral	De acuerdo	Tot. acuerdo	% Acuerdo	% Desacuerdo
P1. Comprendí adecuadamente los conceptos teóricos	8.3 %	19.4 %	27.8 %	36.1 %	8.3 %	44.4 %	27.8 %
P2. Las clases magistrales fueron útiles	0.0 %	8.3 %	36.1 %	33.3 %	22.2 %	55.6 %	8.3 %
P3. Me siento preparado para aplicar los conceptos	8.3 %	36.1 %	36.1 %	19.4 %	0.0 %	19.4 %	44.4 %
P4. Tuve dificultades con conceptos abstractos	2.8 %	13.9 %	5.6 %	52.8 %	25.0 %	77.8 %	16.7 %
P5. Racket y EOPL facilitaron la comprensión	13.9 %	25.0 %	41.7 %	19.4 %	0.0 %	19.4 %	38.9 %
P6. Fue fácil encontrar recursos externos	16.7 %	44.4 %	22.2 %	5.6 %	11.1 %	16.7 %	61.1 %
P7. Recursos adicionales habrían mejorado mi aprendizaje	0.0 %	2.8 %	2.8 %	38.9 %	55.6 %	94.4 %	2.8 %

Fuente: elaboración propia.

Los resultados permiten identificar cuatro hallazgos clave:

Brecha entre comprensión teórica y aplicación práctica : Aunque el 44.4 % de los estudiantes se mostraron de acuerdo en haber comprendido adecuadamente los conceptos teóricos (P1), solo el 19.4 % se sintieron preparados para aplicarlos en otros cursos o proyectos (P3). Esta diferencia de puntos porcentuales revela que la comprensión teórica de los contenidos no se traduce necesariamente en una comprensión funcional que permita su uso autónomo. Adicionalmente, el porcentaje de respuestas neutrales en P3 (36.1 %) es significativamente mayor que en P1 (27.8 %), lo que sugiere que muchos estudiantes no tienen una opinión clara sobre su preparación para aplicar los conceptos, lo que refuerza la idea de una brecha entre teoría y práctica.

Alta incidencia de dificultades con conceptos abstractos : El ítem P4 concentra el mayor porcentaje de acuerdo de toda la encuesta, el 77.8 % de los participantes reportó haber tenido dificultades para comprender conceptos como entornos, estado o secuenciación. Este dato es especialmente significativo porque estos conceptos corresponden directamente al segundo resultado de aprendizaje (R.A.2) del microcurrículo de FLP, el cual concentra el 50 % del total de la nota de la asignatura.

Percepción negativa o neutra de Racket y EOPL como herramientas de aprendizaje : El ítem P5 muestra que solo el 19.4 % de los estudiantes consideró que Racket y EOPL facilitaron su comprensión, mientras que el 38.9 % estuvo en desacuerdo y el 41.7 % restante se mantuvo neutral. Este resultado sugiere que, para la mayoría de los participantes, las herramientas actuales no representan un apoyo claro al aprendizaje, lo que se alinea con la teoría de la carga cognitiva de Sweller, donde la sintaxis de Racket actúa como una carga extrínseca que satura la memoria de trabajo [9] y obstaculiza la comprensión semántica al obligar al estudiante a agotar sus recursos mentales en el descifrado visual del código [10].

Escasez de recursos externos y necesidad de herramientas adicionales : El 61.1 % de los estudiantes indicó que no fue fácil encontrar recursos externos de apoyo (P6), lo que apunta a un vacío en la disponibilidad de material complementario específico para los contenidos de la asignatura. Asimismo, el 94.4 % de los encuestados consideró que recursos adicionales habrían mejorado su aprendizaje de manera significativa (P7), constituyendo el dato de mayor consenso en toda la encuesta.

3.2.2. Resultados cualitativos

Del análisis temático de las recomendaciones proporcionadas por los estudiantes emergen cuatro categorías recurrentes:

Necesidad de experimentación con retroalimentación inmediata. Varios participantes señalaron que el principal obstáculo no es la complejidad conceptual en sí, sino la imposibilidad de explorar los conceptos en tiempo real con acompañamiento. Un estudiante sintetizó esta situación con precisión: *“me gustaría poder tener al profesor en vivo para una pregunta, porque las dudas surgen a medida que uno va haciendo y practicando [...] me quedó el vacío de un diagrama de ambientes”*. Otro participante expresó la misma necesidad de forma más directa: *“crear espacios donde los estudiantes puedan experimentar con Racket y la librería EOPL en tiempo real, con retroalimentación inmediata”*.

Carga cognitiva de Racket como obstáculo al aprendizaje semántico. La fricción generada por la sintaxis de Racket fue identificada de forma explícita por varios estudiantes como un factor que desplaza el foco del aprendizaje. El testimonio más detallado al respecto señala: *“el principal obstáculo del curso es la carga cognitiva que impone Racket. [...] Los nombres de funciones internas (como *car*, *cdr* o *map*) y la estructura visual plana terminan desplazando el foco de la semántica de lenguajes hacia la resolución de errores sintácticos”*.

Limitaciones de la metodología de aula invertida. Cuatro estudiantes mencionaron explícitamente la metodología de aula invertida como un factor de dificultad. Sus comentarios señalan que este modelo, si bien puede ser efectivo en otros contextos, resulta especialmente problemático para una asignatura con alto nivel de abstracción, donde la orientación directa del docente durante la exposición inicial de los conceptos es percibida como necesaria: *“con aula invertida es muy difícil entender”*; *“mayor explicación, resolver ejercicios desde cero con estudiantes”*.

Complejidad acumulada en los temas del último corte. Dos participantes señalaron que los temas de mayor dificultad se concentran en el tramo final del semestre, los cuales están relacionados con estado, paso de parámetros y tipos. Además, el ritmo de avance no permite profundizar en ellos adecuadamente: *“tomar más tiempo para la enseñanza de los temas más complejos que se ven en el último corte”*.

3.3. Análisis de la encuesta a docentes

La encuesta fue realizada por cinco docentes del programa identificados como D1 a D5. El instrumento segmentó los bloques temáticos, permitiendo que los cinco diligenciaran el bloque de percepción

general; D1 y D5 respondieron lo relacionado a la programación imperativa y programación orientada a objetos, teniendo en cuenta que dictan en primero y segundo semestre. Por otro lado, D2, D3 y D4 realizaron el bloque de programación lógica y paradigmas avanzados, lo que corresponde a quinto semestre en adelante. Si bien, el perfil directamente relacionado con FLP es el de los docentes de semestres avanzados, las respuestas de D1 y D5 permiten caracterizar las bases con las que los estudiantes llegan a la asignatura, lo que es un factor determinante en su capacidad de asimilar contenidos de alta abstracción [8].

3.3.1. Percepción general de los estudiantes

Este bloque obtuvo la respuesta de los cinco docentes y permite construir un diagnóstico transversal del programa. La Tabla 3.2 presenta los resultados, agrupando primero a los docentes de semestres avanzados (D2, D3, D4) y luego a los de primeros semestres (D1, D5), para facilitar la comparación por grupo.

Tabla 3.2: Percepción general de los docentes sobre habilidades de los estudiantes ($n = 5$)

Afirmación	D2 (5°+)	D3 (5°+)	D4 (5°+)	D1 (1°-2°)	D5 (1°-2°)
Demuestran habilidades básicas de pensamiento lógico	De acuerdo	Tot. acuerdo	Tot. acuerdo	Neutral	En desacuerdo
Tienen dificultades para analizar un problema antes de programar	En desacuerdo	De acuerdo	De acuerdo	En desacuerdo	Neutral
Logran depurar errores de manera efectiva	De acuerdo	De acuerdo	Neutral	En desacuerdo	Neutral
Pueden dividir problemas complejos en partes más pequeñas	Tot. acuerdo	Neutral	En desacuerdo	En desacuerdo	Neutral
Comprenden la importancia de la abstracción	Tot. acuerdo	Tot. acuerdo	En desacuerdo	En desacuerdo	Neutral

Fuente: elaboración propia.

La lectura de la Tabla 3.2 revela una brecha de percepción entre grupos. Los docentes de semestres avanzados tienden a valorar positivamente las habilidades de sus estudiantes más que los de primeros semestres, lo cual es un resultado esperable dado que los estudiantes de semestres superiores han superado los filtros académicos iniciales. D1 se muestra consistentemente en desacuerdo en cuatro de los cinco ítems, mientras que D5 se mantiene neutral, ambos describiendo un perfil de estudiante de primeros semestres con habilidades lógicas y de abstracción mínimas.

Además, se observa una variedad de respuestas dentro del grupo avanzado, mientras D2 y D3 coinciden en que los estudiantes de semestre superiores comprenden la importancia de la abstracción, D4 está en desacuerdo con esta afirmación. Lo mismo ocurre con la capacidad de dividir problemas complejos, donde D2 responde "Totalmente de acuerdo" D4 "En desacuerdo". Estas diferencias dentro del mismo grupo de docentes indica que las habilidades de abstracción y descomposición de problemas no están consolidadas de manera uniforme en los estudiantes.

Esta observación es coherente con lo expuesto por Zagami, quien señala que los estudiantes enfrentan una alta demanda cognitiva al intentar procesar conceptos abstractos simultáneamente con habilidades prácticas; cuando la capacidad de la memoria de trabajo se ve excedida por esta multiplicidad de elementos, se generan bloqueos y sentimientos de frustración en el aprendiz [8]. Asimismo, se

reconoce que la naturaleza intrínsecamente abstracta de los algoritmos y construcciones teóricas en esta disciplina representa un desafío que dificulta la comprensión intuitiva [3].

3.3.2. Dificultades con paradigmas avanzados

Los docentes D2, D3 y D4 respondieron el bloque sobre los paradigmas directamente relacionados con los contenidos de FLP. La Tabla 3.3 muestra sus respuestas, evidenciando una percepción más positiva en D2 y D3, mientras que D4 se muestra más crítico, especialmente en relación con la capacidad de los estudiantes para seleccionar el paradigma adecuado para resolver un problema.

Tabla 3.3: Percepción de los docentes de semestres avanzados sobre programación lógica y paradigmas avanzados ($n = 3$)

Afirmación	D2 (5°+)	D3 (5°+)	D4 (5°+)
Comprenden los conceptos fundamentales de programación lógica	Tot. acuerdo	Tot. acuerdo	Tot. acuerdo
Tienen dificultades con el razonamiento lógico aplicado a la programación	De acuerdo	De acuerdo	En desacuerdo
Logran modelar problemas usando reglas lógicas	Tot. acuerdo	Neutral	De acuerdo
Comprenden la diferencia entre los distintos paradigmas	Tot. acuerdo	De acuerdo	En desacuerdo
Pueden seleccionar el paradigma adecuado para resolver un problema	De acuerdo	De acuerdo	Tot. en desacuerdo

Fuente: elaboración propia.

El único ítem en el que los tres docentes coinciden es la comprensión de los conceptos fundamentales de programación lógica, donde todos respondieron "Totalmente de acuerdo". Sin embargo, esta percepción contrasta con el resto de las respuestas lo que sugiere que, saber qué es la programación lógica no implica saber aplicarla para resolver problemas.

Las opiniones también difieren respecto al razonamiento lógico aplicado a la programación. D2 y D3 consideran que los estudiantes presentan dificultades en este aspecto, mientras que D4 no lo percibe así. Estas diferencias pueden estar relacionadas con el contexto y el tipo de actividades realizados, ya que en cursos más guiados, algunas dificultades pueden pasar desapercibidas, pero, en asignaturas como FLP, donde los estudiantes deben construir soluciones con mayor autonomía, estas limitaciones resultan más evidentes.

Así mismo, la capacidad de comprender las diferencias entre paradigmas y seleccionar el adecuado para un problema concentra la mayor dispersión del bloque, debido a que D2 y D3 tienen una percepción positiva, pero, D4 está respectivamente "En desacuerdo" y "Totalmente en desacuerdo". Esta percepción negativa demuestra el salto cognitivo que representa abandonar el paradigma imperativo para adoptar el paradigma funcional [11], dificultad que se agrava cuando el lenguaje utilizado impone una carga sintáctica adicional.

3.4. Revisión del microcurrículo

El microcurrículo de la asignatura FLP organiza sus contenidos en torno a tres competencias específicas y una competencia genérica, articuladas a través de tres resultados de aprendizaje y veinte

indicadores de logro distribuidos con pesos diferenciados en las actividades de evaluación.

El **Resultado de Aprendizaje 1 (R.A.1)**, con un peso del 40 % sobre la nota total, abarca la construcción de datos recursivos mediante gramáticas BNF, el manejo del cálculo lambda, alcance de variables, implementación de tipos abstractos de datos y la construcción de árboles de sintaxis abstracta. Este R.A. sienta las bases formales sobre las que se construye todo el trabajo posterior de la asignatura.

El **Resultado de Aprendizaje 2 (R.A.2)**, con el mayor peso de la asignatura (50 %), se centra en la implementación de lenguajes con ambientes de variables, el recorrido del AST, la construcción de intérpretes básicos y la implementación de conceptos como condicionales, clausuras, estado y paso de parámetros. La profundidad de este resultado, sumado a su peso evaluativo, lo convierte en el eje central de la asignatura y en la fuente principal de dificultades reportadas por los estudiantes.

El **Resultado de Aprendizaje 3 (R.A.3)**, con un peso del 5 %, tiene un carácter más conceptual, puesto que se enfoca en que los estudiantes comprendan las diferencias entre compilación e interpretación identificando sus beneficios y limitaciones.

La **Competencia Genérica (C.G.2)**, con un peso del 5 %, integra varios indicadores del R.A.2 en la construcción de un intérprete completo evaluado en el proyecto final de la asignatura.

3.5. Selección de conceptos base para el prototipo

A partir de las dificultades reportadas por los estudiantes, las percepciones del cuerpo docente y la revisión del microcurrículo, se seleccionaron cinco indicadores de logro como base conceptual para la biblioteca de casos predefinidos del prototipo. El criterio de selección combinó tres factores, entre los que se encuentran el nivel de dificultad reportado por los estudiantes, el peso evaluativo en el microcurrículo y la viabilidad de representación visual e interactiva en una herramienta web.

La Tabla 3.4 presenta los indicadores de logro seleccionados, junto con su relación con los resultados de aprendizaje y una justificación para su inclusión en el prototipo.

Tabla 3.4: Indicadores de logro seleccionados como base para el prototipo

ID	Indicador de logro	R.A.	Justificación
IL 1.1.6	Construye un árbol de sintaxis abstracta a partir de una representación concreta en una gramática BNF	R.A.1	El AST es la estructura central del proceso de interpretación. Los estudiantes reportan dificultades para visualizar cómo una cadena de texto se transforma en una representación jerárquica; su visualización dinámica es la funcionalidad más distintiva del prototipo.
IL 1.2.3	Construye intérpretes básicos para dos conjuntos de valores (valor expresado, valor denotado)	R.A.2	Corresponde al primer nivel de construcción de intérpretes, donde se establece la relación entre la gramática, el AST y la evaluación. Es el punto de entrada natural para comprender el proceso completo de interpretación.
IL 1.2.6	Implementa lenguajes de programación con el concepto de estado asociado al manejo de ambientes para variables en lenguajes imperativos	R.A.2	El concepto de estado y su representación mediante ambientes fue identificado como el de mayor dificultad por los estudiantes (77.8 % reportó dificultades con entornos y estado). La representación visual de cómo evoluciona el ambiente durante la ejecución es uno de los aportes centrales del prototipo.
IL 1.2.7	Comprende la diferencia entre ligadura y asignación de variables	R.A.2	La distinción entre ligadura y asignación es conceptualmente sutil y frecuentemente confundida. Su representación visual mediante ambientes permite hacer explícita la diferencia que el código por sí solo no comunica con claridad.
IL 1.3.1	Identifica los componentes necesarios de un intérprete y un compilador	R.A.3	Provee el marco conceptual global que da sentido al resto de los contenidos. Comprender la arquitectura de un intérprete permite al estudiante situar cada concepto específico dentro de un proceso coherente.

Fuente: elaboración propia a partir del microcurrículo de la asignatura (código 750017C).

Estos cinco indicadores cubren los dos ejes de dificultad identificados en el diagnóstico. Por un lado, el eje sintáctico estructural, representado por IL 1.1.6, que corresponde a la construcción del AST. Por otro lado, el eje semántico-evaluativo, representado por IL 1.2.3, 1.2.6 y 1.2.7, referentes a la construcción del intérprete y el manejo del ambiente, complementado por el eje conceptual arquitectónico (IL 1.3.1).

Capítulo 4

Diseño del prototipo FLP Viewer

El presente capítulo documenta los resultados correspondientes al diseño del prototipo web propuesto en este trabajo, denominado **FLP Viewer**. El nombre constituye un juego de palabras que combina la sigla FLP, empleada a lo largo de este documento para referirse a la asignatura *Fundamentos de Interpretación y Compilación de Lenguajes de Programación*, con el término en inglés *viewer* (visualizador), que alude al propósito central de la herramienta, esto es, permitir al estudiante observar y explorar el comportamiento de un intérprete durante su ejecución.

En primer lugar, se presenta la especificación de requisitos del sistema, derivada del diagnóstico realizado en el capítulo anterior y de los indicadores de logro seleccionados como base conceptual del prototipo. En segundo lugar, se describe la arquitectura adoptada, la selección y justificación de tecnologías, los prototipos de interfaz de usuario y las estrategias de visualización implementadas para apoyar la comprensión de los contenidos de la asignatura.

4.1. Especificación de requisitos del sistema

4.1.1. Enfoque metodológico

La identificación y documentación de los requisitos del sistema se realizó siguiendo las directrices del estándar IEEE 830 [12] para la especificación de requisitos de software, adaptado al contexto educativo del proyecto. Los requisitos fueron derivados a partir de tres fuentes complementarias. En primer lugar, las dificultades conceptuales identificadas en el diagnóstico con estudiantes y docentes (Capítulo 3); en segundo lugar, los indicadores de logro del microcurrículo de la asignatura (Tabla 3.4); y en tercer lugar, las características de calidad de software definidas en la norma ISO/IEC 25010 [13], que orientó la formulación de los requisitos no funcionales.

Cada requisito fue clasificado por prioridad (Alta, Media) en función de su relevancia para los objetivos de aprendizaje identificados y de su viabilidad de implementación en el alcance del proyecto. Los criterios de aceptación asociados a cada requisito fueron formulados de manera verificable, haciendo referencia al comportamiento concreto del sistema implementado. La especificación completa se encuentra en el Anexo D.

4.1.2. Requisitos funcionales

Se definieron ocho requisitos funcionales que cubren las capacidades esenciales del prototipo. La Tabla 4.1 presenta el resumen de los mismos con su prioridad y una descripción concisa.

Tabla 4.1: Resumen de requisitos funcionales del sistema

ID	Nombre	Descripción resumida	Prioridad
RF-01	Generación del AST	Procesar el programa del usuario con la gramática definida y producir el AST correspondiente.	Alta
RF-02	Visualización del AST	Renderizar el AST generado de forma jerárquica e interactiva en un panel dedicado.	Alta
RF-03	Ejecución y evaluación	Invocar el intérprete Racket sobre los archivos del editor y mostrar el resultado en la consola.	Alta
RF-04	Representación del ambiente	Mostrar el ambiente de evaluación como estructura jerárquica de frames actualizada tras cada ejecución.	Alta
RF-05	Biblioteca de ejemplos	Ofrecer un conjunto de programas predefinidos alineados con los indicadores de logro de la asignatura.	Media
RF-06	Validación de sintaxis Racket	Detectar y reportar errores léxicos y sintácticos en la gramática BNF y en el programa del usuario.	Alta
RF-07	Visualización del intérprete	Presentar los archivos del intérprete generado (gramática, ambiente, evaluador) en un editor organizado por pestañas con secciones protegidas.	Media
RF-08	Plantilla base y exportación	Proveer una plantilla editable del intérprete con secciones bloqueadas y permitir la descarga del proyecto en formato .zip.	Alta

Fuente: elaboración propia. Especificación completa en Anexo D.

4.1.3. Requisitos no funcionales

Se definieron cinco requisitos no funcionales que establecen las condiciones de calidad del sistema. La Tabla 4.2 presenta el resumen correspondiente.

Tabla 4.2: Resumen de requisitos no funcionales del sistema

ID	Nombre	Descripción resumida	Prioridad
RNF-01	Usabilidad	La interfaz debe ser operada sin instrucción previa por estudiantes sin experiencia avanzada en compiladores.	Alta
RNF-02	Rendimiento	La generación del AST y la evaluación del programa deben completarse en menos de 5 segundos en condiciones normales.	Alta
RNF-03	Accesibilidad web	El sistema debe ejecutarse en navegadores modernos sin instalación adicional ni autenticación.	Alta
RNF-04	Mantenibilidad	El código debe estar organizado en módulos independientes con separación explícita de tipos, lógica y presentación.	Media
RNF-05	Escalabilidad	La arquitectura debe permitir incorporar nuevos ejemplos, gramáticas o características sin afectar las funcionalidades existentes.	Media

Fuente: elaboración propia. Especificación completa en Anexo D.

4.2. Diseño del prototipo

4.2.1. Arquitectura del sistema

La arquitectura del prototipo se estructuró en tres capas principales: infraestructura de despliegue, servidor y cliente. Esta separación de responsabilidades permite que el procesamiento computacionalmente intensivo (la ejecución de Racket) ocurra en el servidor, mientras que la interfaz de usuario opera íntegramente en el navegador sin necesidad de recargas de página [14].

Las decisiones que dieron lugar a esta arquitectura estuvieron guiadas por tres criterios principales: simplicidad operativa, aislamiento de la ejecución de código arbitrario y minimización de la latencia percibida por el usuario. A partir de estos criterios se definió la distribución de responsabilidades entre los componentes y el modelo de despliegue del sistema.

Con respecto a la simplicidad operativa, se optó por una arquitectura monolítica de servidor único en lugar de una arquitectura de microservicios. Fowler y Lewis señalan que los microservicios introducen una complejidad operativa significativa, relacionada con la coordinación entre servicios, la trazabilidad distribuida y la gestión de fallos, que solo se justifica cuando la escala o la independencia de despliegue de los componentes lo requieren [15]. Dado que el prototipo cuenta con un equipo de desarrollo reducido y un conjunto acotado de funcionalidades, la arquitectura monolítica resulta más adecuada para el alcance del proyecto.

Respecto al aislamiento de la ejecución de código arbitrario, la ejecución de Racket se delega a un subproceso del servidor de aplicación en lugar de separarse como un servicio independiente. Cada invocación se ejecuta en un proceso con un tiempo límite estricto y un directorio temporal propio. De esta manera, un programa con un bucle infinito o un comportamiento inesperado no compromete la estabilidad del servidor principal [16].

Finalmente, para reducir la latencia percibida por el usuario, el pipeline de transformación de gramática se ubicó íntegramente en el cliente. Esta decisión acerca el cómputo al punto de generación de los datos y elimina la latencia de red asociada a las validaciones inmediatas [14]. Además, esta estrategia es viable porque la carga computacional del pipeline es baja y no requiere acceso a recursos

del servidor.

4.2.1.1. Infraestructura de despliegue

La infraestructura de despliegue se compone de dos elementos. El primero es un reverse proxy¹, encargado de recibir las solicitudes entrantes y redirigirlas al servidor de aplicación, centralizando así el punto de entrada al sistema. El segundo es el propio servidor de aplicación, que incorpora el intérprete de Racket como dependencia del entorno de ejecución. Esta configuración garantiza que el prototipo sea reproducible independientemente del sistema operativo del servidor anfitrión. La selección de las tecnologías concretas que materializan esta estructura se detalla en la sección 4.2.2.

4.2.1.2. Capa de servidor

La capa de servidor asume dos responsabilidades principales: el renderizado inicial de la interfaz y la ejecución del intérprete. Durante el renderizado inicial, los datos estáticos del sistema, como los ejemplos predefinidos y los contenidos de ayuda, se cargan desde el sistema de archivos y se transmiten directamente a la capa cliente. De esta manera, se evita la necesidad de realizar peticiones adicionales una vez cargada la página.

La segunda responsabilidad corresponde a la ejecución de programas en Racket. Para ello, el servidor expone un punto de acceso interno que recibe los archivos generados por el editor y la expresión a evaluar. A continuación, lanza el proceso de Racket de forma aislada, aplicando un tiempo límite configurable para su ejecución, y devuelve al cliente el resultado estructurado como una secuencia de pasos.

4.2.1.3. Capa de cliente

La capa cliente se diseñó alrededor de un componente orquestador central encargado de mantener el estado global de la sesión de trabajo y coordinar la comunicación entre los distintos paneles de la interfaz. Este componente centraliza las transiciones de estado, incluyendo el inicio y fin de sesión, la recepción de resultados y la navegación paso a paso por las evaluaciones. Gracias a ello, cada panel de visualización puede implementarse como un componente stateless² que recibe únicamente la información necesaria para su renderizado.

¹Un *reverse proxy* es un servidor interpuesto entre el cliente y el servidor de aplicación que recibe las solicitudes entrantes y las reenvía al servicio interno correspondiente, ocultando los detalles de la infraestructura backend y permitiendo aplicar reglas de seguridad, balanceo de carga o compresión de manera centralizada.

²Un componente *stateless* (sin estado) es aquel que no almacena información propia entre renders; su salida depende exclusivamente de los datos que recibe como entrada, lo que facilita su prueba y reutilización.

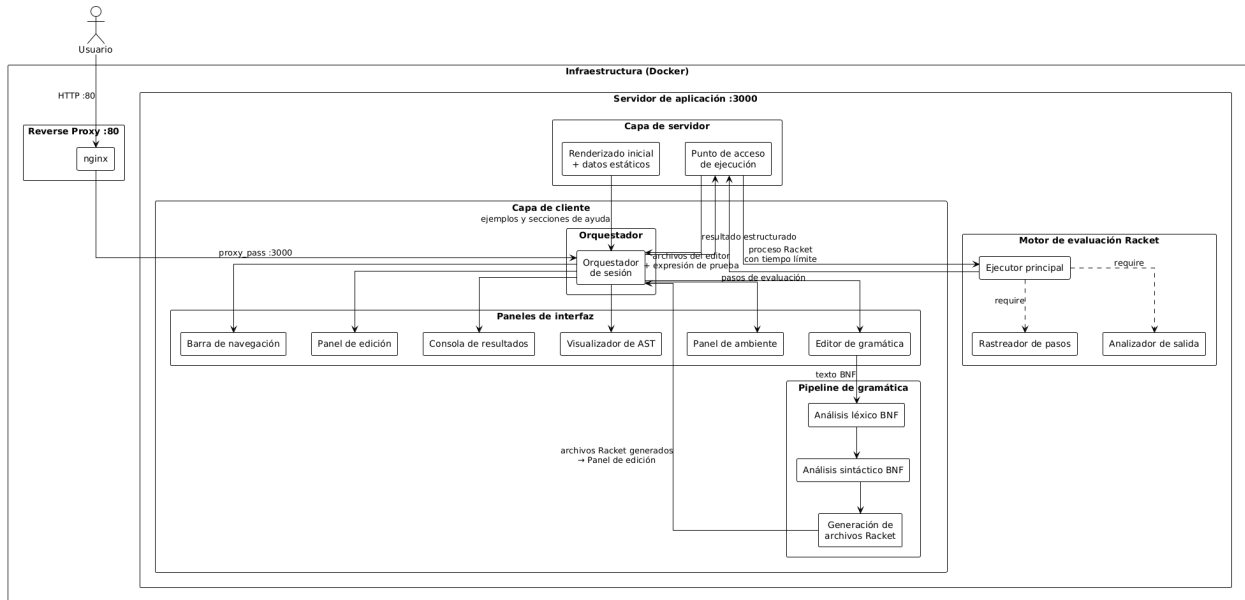


Figura 4.1: Diagrama de arquitectura del sistema

Fuente: Elaboración propia

4.2.1.4. Pipeline de gramática

Dentro de la arquitectura cliente, el pipeline de transformación de gramática constituye un subsistema encargado de procesar las especificaciones definidas por el estudiante sin intervención del servidor. Se diseñó como una cadena de etapas independientes que comprende el análisis léxico de la notación BNF, el análisis sintáctico para la construcción de una representación intermedia de la gramática y la generación de los archivos del intérprete a partir de dicha representación.

La separación en etapas independientes permite identificar y reportar errores de forma precisa en el nivel donde se originan (léxico o sintáctico) y favorece una organización modular del proceso de transformación [17]. Además, esta estructura facilita la incorporación de nuevas etapas o transformaciones sin alterar significativamente los componentes existentes.

Al ejecutarse íntegramente en el cliente, el pipeline proporciona retroalimentación inmediata sobre errores en la gramática sin depender de intercambios con el servidor. Esta característica reduce el tiempo de espera entre la acción del estudiante y la respuesta del sistema, favorece la continuidad de la tarea y contribuye a disminuir la carga cognitiva extrínseca [9].

4.2.2. Selección de tecnologías

La selección de tecnologías se realizó evaluando cuatro criterios. Se consideraron la curva de aprendizaje del equipo de desarrollo, la disponibilidad de documentación, la viabilidad de implementación de las funcionalidades requeridas y la compatibilidad entre las tecnologías seleccionadas. La Tabla 4.3 presenta la justificación de cada elección.

Tabla 4.3: Tecnologías seleccionadas y justificación

Tecnología	Versión	Justificación
Next.js [18]	16.2.6	Framework basado en React que integra renderizado en servidor y rutas de API en una única plataforma. Su adopción permite implementar la arquitectura propuesta sin requerir servicios backend adicionales.
React [19]	19.2	Biblioteca para la construcción de interfaces de usuario basada en componentes. Su modelo declarativo facilita la implementación de la interfaz interactiva y la separación de responsabilidades entre los distintos paneles del sistema.
TypeScript [20]	5.9	Superset tipado de JavaScript que permite definir contratos explícitos entre módulos mediante interfaces y tipos. Su utilización contribuye a reducir errores durante el desarrollo y a mejorar la mantenibilidad del código.
Monaco Editor [21]	0.55	Componente de edición utilizado por Visual Studio Code y disponible como librería independiente. Proporciona las capacidades necesarias para implementar el editor interactivo, incluyendo la protección de regiones de código generadas automáticamente.
Tailwind CSS [22]	4	Framework de estilos utilitarios que facilita la construcción de interfaces consistentes mediante clases reutilizables. Su enfoque permite mantener los estilos próximos a los componentes que los utilizan y simplifica el desarrollo de la capa de presentación.
Racket [4]	9.x	Lenguaje utilizado en la asignatura FLP y plataforma de ejecución de la librería EOPL. Su adopción garantiza la compatibilidad entre el código generado por el prototipo y el entorno académico empleado por los estudiantes.
Docker [23]	—	Plataforma de contenedores utilizada para encapsular el entorno de ejecución. Permite garantizar la reproducibilidad del despliegue e incorporar Racket como dependencia sin requerir su instalación en el sistema anfitrión.
nginx	alpine	Servidor web empleado como reverse proxy delante del servidor de aplicación. Centraliza el acceso al sistema y permite gestionar aspectos de infraestructura como compresión, redirección de solicitudes y configuración de seguridad.
pnpm	10	Gestor de paquetes para el ecosistema JavaScript con soporte de almacenamiento eficiente de dependencias. Su utilización reduce los tiempos de instalación y construcción del proyecto en el entorno de despliegue.

Fuente: elaboración propia.

La selección realizada refleja las decisiones arquitectónicas descritas en la sección anterior. Next.js, React y TypeScript conforman la base de la capa cliente y del servidor de aplicación, mientras que Monaco Editor proporciona las capacidades de edición requeridas por el entorno interactivo. Por su parte, Racket constituye el motor de evaluación del prototipo y Docker garantiza la reproducibilidad de su entorno de ejecución. Finalmente, nginx complementa la infraestructura de despliegue como punto de entrada único al sistema.

4.2.3. Prototipos de interfaz de usuario

La distribución de la interfaz se diseñó con base en los principios de economía de la atención [24] y en los hallazgos del diagnóstico, que evidenciaron que la fricción visual de las herramientas actuales

desplaza la atención del estudiante desde la semántica del lenguaje hacia la resolución de errores sintácticos. Para mitigar este efecto, la interfaz se organizó de manera que las tareas principales del estudiante, que incluyen la definición de la gramática, la edición del intérprete, la ejecución de programas y el análisis de resultados, siguieran un recorrido visual coherente dentro de una única pantalla, minimizando los cambios de contexto.

La literatura sobre herramientas educativas para la enseñanza de compiladores e intérpretes destaca la importancia de presentar simultáneamente múltiples representaciones del proceso de ejecución, permitiendo que el estudiante relacione el código fuente con las estructuras internas generadas durante el procesamiento [3,5,6]. Asimismo, los principios de carga cognitiva sugieren que la integración de información relevante dentro de un mismo espacio de trabajo reduce el esfuerzo necesario para coordinar fuentes dispersas de información [9]. Por esta razón, se optó por una interfaz multipanel en la que las representaciones principales del proceso de interpretación permanecen visibles de manera simultánea.

La interfaz principal se estructura en cinco zonas funcionales, cuya distribución se presenta en el wireframe³ de la Figura 4.2. La organización propuesta define una configuración inicial de los paneles, permitiendo al estudiante modificar dinámicamente el espacio asignado a cada zona durante la interacción. De esta manera, es posible redistribuir el área disponible entre el editor, las visualizaciones del AST y del ambiente, y la consola de salida según las necesidades de la tarea en curso. Esta flexibilidad permite priorizar las representaciones más relevantes en cada momento, manteniendo una visión integrada del proceso de interpretación y contribuyendo a la usabilidad del sistema [7].

1. **Barra de navegación** (zona superior): concentra el acceso a las funciones globales del sistema, incluyendo la biblioteca de ejemplos, el editor de gramática y los controles asociados a la sesión de ejecución.
2. **Panel de edición** (zona izquierda): alberga el editor de código organizado en múltiples pestañas, desde donde el estudiante interactúa con los distintos archivos que conforman el intérprete.
3. **Panel del AST** (zona derecha superior): presenta la representación jerárquica del árbol de sintaxis abstracta generado durante la evaluación de expresiones.
4. **Panel de ambiente** (zona derecha inferior): visualiza el estado del entorno de ejecución mediante una estructura de frames anidados que muestra los bindings activos en cada nivel de alcance.
5. **Panel de consola** (zona inferior): concentra la interacción textual con el intérprete, permitiendo introducir expresiones y consultar los resultados producidos durante la ejecución.

³Un *wireframe* es un prototipo de baja fidelidad utilizado en el diseño de interfaces para representar la estructura, distribución espacial y flujo de interacción de una aplicación, sin incorporar elementos visuales finales ni detalles de implementación.

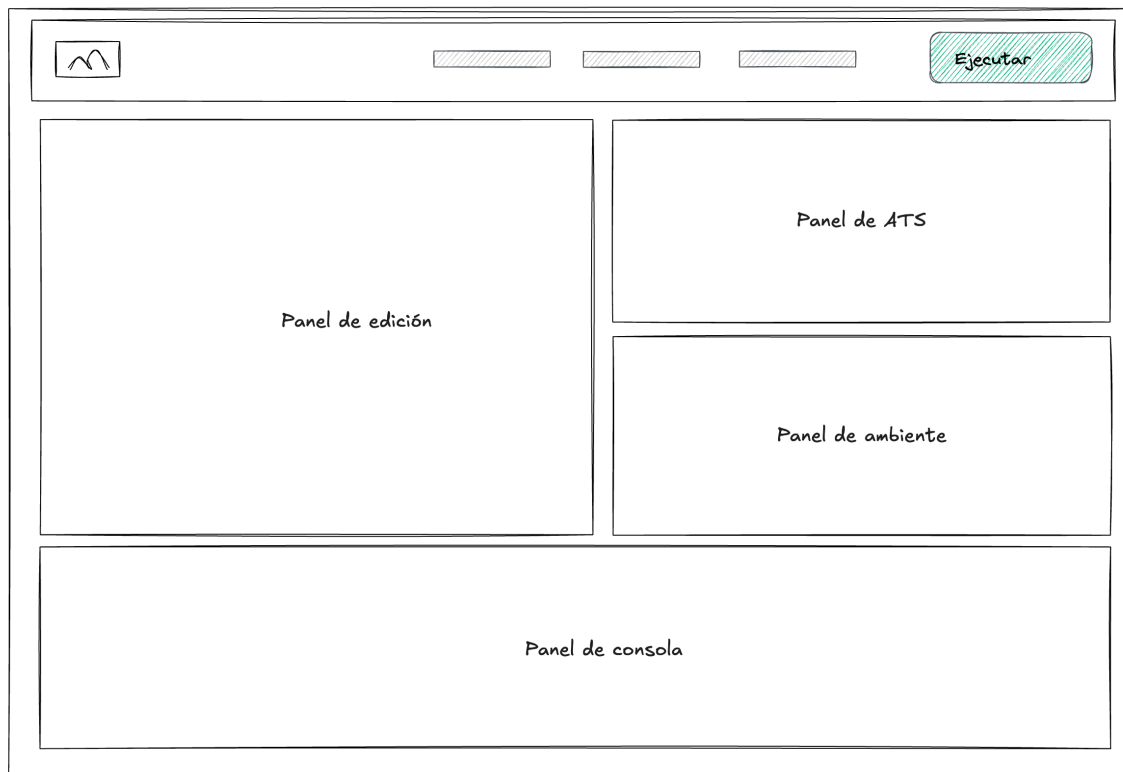


Figura 4.2: Wireframe de la interfaz principal del prototipo FLP Viewer

Fuente: Elaboración propia

Adicionalmente, se diseñó un modal de gramática (Figura 4.3) que permite al estudiante ingresar la especificación léxica y la gramática en notación BNF en dos áreas de texto diferenciadas. Al confirmar la definición, el sistema genera los archivos del intérprete y los carga en el panel de edición. Este flujo de interacción mantiene la gramática y el intérprete en un vínculo explícito y visible para el estudiante, de forma que la relación entre la especificación formal y el código generado pueda observarse directamente. Esta decisión responde a la necesidad de hacer visibles procesos que normalmente permanecen ocultos en los entornos de desarrollo convencionales, una estrategia identificada como beneficiosa para la enseñanza de compiladores e intérpretes [3, 5].



Figura 4.3: Wireframe del modal de definición de gramática BNF

Fuente: Elaboración propia

4.2.4. Estrategias de visualización

El diseño de las estrategias de visualización responde directamente a los dos ejes de dificultad identificados en el diagnóstico: el eje sintáctico-estructural, asociado a la comprensión del AST, y el eje semántico-evaluativo, asociado al seguimiento del estado durante la ejecución [6]. En cada caso, las decisiones de diseño se orientaron a maximizar la correspondencia entre las representaciones visuales y los modelos conceptuales presentados en la asignatura.

El AST se renderiza como un árbol jerárquico interactivo en el que cada nodo muestra el nombre del tipo de producción que lo originó y, cuando corresponde, el valor léxico del token asociado. Esta representación hace visible la correspondencia entre la notación BNF definida por el estudiante y la estructura interna que el parser produce. Sorva et al. señalan que la efectividad de este tipo de herramientas está directamente relacionada con el grado en que la representación visual se corresponde con el modelo conceptual que el estudiante debe construir [25]. Para apoyar esa lectura, los nodos se diferencian visualmente según su categoría semántica, lo que es consistente con los hallazgos de Stamenkovic et al. [3] y Jovanovic et al. [6]. Asimismo, los nodos admiten expansión y colapso interactivo y, por defecto, solo el nivel raíz se expande al cargar un nuevo árbol, de modo que el estudiante parte de una vista de alto nivel y profundiza progresivamente según su interés, en coherencia con los principios de carga cognitiva [9].

El ambiente de ejecución se representa como una cadena de frames enlazados, donde cada frame expone las ligaduras activas en ese nivel de alcance con su nombre, valor y tipo. La disposición hace visible la relación de extensión entre los entornos, de modo que el estudiante puede seguir la cadena desde el entorno inicial hasta el más reciente. Esta representación busca hacer observable

la diferencia entre ligadura y asignación (IL 1.2.7) y la evolución del ambiente con cada llamada a función (IL 1.2.6), abordando directamente la dificultad reportada por el 77.8 % de los estudiantes encuestados. La disposición en cadena sigue el modelo conceptual descrito por Friedman y Wand [4], favoreciendo la correspondencia entre la teoría presentada en clase y la visualización ofrecida por la herramienta, lo cual es relevante porque representaciones que no coinciden con el modelo enseñado pueden inducir concepciones erróneas difíciles de corregir [26].

El modo de evaluación paso a paso permite que, cuando el estudiante envía múltiples expresiones al intérprete, pueda navegar por los resultados de cada una de forma individual. En cada paso, el AST y el ambiente se actualizan para reflejar el estado del intérprete tras esa evaluación, haciendo visible cómo evolucionan el árbol y las ligaduras de una expresión a la siguiente. Este mecanismo es análogo al *stepper* algebraico propuesto por Clements, Flatt y Felleisen [27] y sigue el enfoque de herramientas como Python Tutor [26]. Desde la perspectiva de la carga cognitiva, la presentación controlada de un paso a la vez reduce la presión sobre la memoria de trabajo [9].

Finalmente, la consola de salida diferencia mediante color y prefijo textual cinco categorías de mensaje: entrada del usuario, valor de retorno, información, advertencia y error. Esta distinción permite al estudiante identificar la naturaleza de cada mensaje sin necesidad de leer su contenido, reduciendo el esfuerzo de interpretación de la retroalimentación. La codificación cromática está respaldada por Hannebauer et al. [28], y la decisión de no exponer directamente la salida nativa de Racket responde a los principios de usabilidad en tecnología educativa que recomiendan facilitar la interpretación de la información relevante durante la interacción [7].

Bibliografía

- [1] Q. Liu, W. Zhao, D. Liu, and J. Ji, “Evaluation of situational case-based teaching technique in engineering education,” in *Proceedings of the ACM Turing 50th Celebration Conference - China*, ACM TURC ’17, pp. 1–7, Association for Computing Machinery, May 2017.
- [2] E. A. Navas-López, “Modular and didactic compiler design with xml inter-phases communication,” *International Journal of Computer Science, Engineering and Information Technology (IJCEIT)*, vol. 12, pp. 1–15, February 2022.
- [3] S. Stamenković and N. Jovanović, “A web-based educational system for teaching compilers,” *IEEE Transactions on Learning Technologies*, vol. 17, pp. 143–156, 2024.
- [4] D. P. Friedman and M. Wand, *Essentials of Programming Languages, 3rd Edition*. The MIT Press, 3 ed., 2008.
- [5] L. Spigariol, J. Bono, and F. S. Guijarro, “Aprendiendo a desarrollar un intérprete de un lenguaje de programación funcional,” in *Actas del XXVIII Congreso Argentino de Ciencias de la Computación (CACIC)*, (La Rioja, Argentina), pp. 166–175, Universidad Tecnológica Nacional; Universidad Nacional de General San Martín; Universidad de Buenos Aires; Universidad Nacional de Hurlingham, Red de Universidades con Carreras en Informática, Oct. 2023.
- [6] N. Jovanović, S. Stamenković, and D. Miljković, “Comvis — interactive simulation environment for compiler learning,” *Computer Applications in Engineering Education*, vol. 30, no. 2, pp. 275–291, 2021.
- [7] J. Lu, M. Schmidt, M. Lee, and R. Huang, “Usability research in educational technology: A state-of-the-art systematic review,” *Educational Technology Research and Development*, vol. 70, no. 6, pp. 1951–1992, 2022.
- [8] J. Zagami, *Cognitive Influences on Learning Programming*, pp. 389–399. Springer, 02 2023.
- [9] J. Sweller, “Cognitive load theory and individual differences,” *Learning and Individual Differences*, vol. 110, p. 102423, 2024.
- [10] C. S. Gordon, “The linguistics of programming,” in *Proceedings of the 2024 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward! ’24)*, (New York, NY, USA), pp. 1–21, ACM, 2024.
- [11] B. Németh, E. Choi, E. Makihara, and H. Iida, “Investigating compilation errors of students learning haskell,” *Trends in Functional Programming in Education*, vol. 295, pp. 52–64, 2019.

- [12] “IEEE recommended practice for software requirements specifications,” Tech. Rep. IEEE Std 830-1998, Institute of Electrical and Electronics Engineers, New York, NY, USA, 1998.
- [13] “Systems and software engineering — systems and software quality requirements and evaluation (SQuaRE) — system and software quality models,” Tech. Rep. ISO/IEC 25010:2011, International Organization for Standardization, Geneva, Switzerland, 2011.
- [14] R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000.
- [15] M. Fowler and J. Lewis, “Microservices: a definition of this new architectural term,” *martin-fowler.com*, March 2014. Consultado en 2025.
- [16] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*. Pearson, 4 ed., 2015.
- [17] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*. Pearson Addison-Wesley, 2 ed., 2006.
- [18] Vercel, “Next.js — the react framework for the web.” <https://nextjs.org>, 2024. Versión 16.2.6.
- [19] Meta Open Source, “React — a javascript library for building user interfaces.” <https://react.dev>, 2024. Versión 19.
- [20] Microsoft, “Typescript — javascript with syntax for types.” <https://www.typescriptlang.org>, 2024. Versión 5.9.
- [21] Microsoft, “Monaco editor.” <https://microsoft.github.io/monaco-editor>, 2024. Versión 0.55.
- [22] Tailwind Labs, “Tailwind CSS — a utility-first CSS framework.” <https://tailwindcss.com>, 2024. Versión 4.
- [23] D. Merkel, “Docker: Lightweight linux containers for consistent development and deployment,” *Linux Journal*, vol. 2014, March 2014.
- [24] D. A. Norman, *The Design of Everyday Things: Revised and Expanded Edition*. New York: Basic Books, 2013.
- [25] J. Sorva, A. Karavirta, and L. Malmi, “A review of generic program visualization systems for introductory programming education,” *ACM Transactions on Computing Education*, vol. 13, no. 4, pp. 1–64, 2013.
- [26] P. J. Guo, “Online Python Tutor: Embeddable web-based program visualization for CS education,” in *Proceedings of the 44th ACM Technical Symposium on Computer Science Education*, pp. 579–584, ACM, 2013.
- [27] J. Clements, M. Flatt, and M. Felleisen, “Modeling an algebraic stepper,” in *Programming Languages and Systems (ESOP 2001)*, vol. 2028 of *Lecture Notes in Computer Science*, pp. 320–334, Springer, 2001.
- [28] C. Hannebauer, M. Hesenius, and V. Gruhn, “Does syntax highlighting help programming novices?,” *Empirical Software Engineering*, vol. 23, no. 4, pp. 1987–2013, 2018.

Anexo A

Microcurrículo de FLP

https://docs.google.com/spreadsheets/d/1loAXsbH8Y6nZ8b7QET8K_-sNE15NTxSg/edit?usp=sharing&ouid=117991041727654766098&rtpof=true&sd=true

Anexo B

Encuesta a estudiantes

<https://docs.google.com/spreadsheets/d/1iAudDGoib0yQa7UZzohLvkJp29nifeoau/edit?usp=sharing&ouid=117991041727654766098&rtpof=true&sd=true>

Anexo C

Encuesta a docentes

https://docs.google.com/spreadsheets/d/1loAXsbH8Y6nZ8b7QET8K_-sNE15NTxSg/edit?usp=sharing&ouid=117991041727654766098&rtpof=true&sd=true

Anexo D

Requisitos del sistema

D.1. Requisitos Funcionales

Código	RF-01
Título	Generación del Árbol de Sintaxis Abstracta (AST)
Descripción	El sistema debe procesar el programa ingresado por el usuario utilizando la gramática previamente definida para generar su correspondiente Árbol de Sintaxis Abstracta (AST).
Prioridad	Alta
Criterios de aceptación	▪ El sistema invoca el intérprete Racket con los archivos del editor y la expresión de entrada; el AST resultante es serializado en formato JSON y transmitido al cliente. ▪ El AST generado es renderizado en el panel ASTViewer reflejando la estructura jerárquica del programa evaluado. ▪ Cuando el programa contiene errores sintácticos o de evaluación, el sistema muestra un mensaje de error en la consola con el prefijo × y nivel visual diferenciado en rojo; el panel ASTViewer conserva el último AST válido. ▪ El AST corresponde fielmente a la última expresión evaluada exitosamente.

Tabla D.1: RF-01: Generación del AST

Código	RF-02
Título	Visualización gráfica e interactiva del AST
Descripción	El sistema debe mostrar el árbol de sintaxis abstracta generado de forma gráfica e interactiva, permitiendo la exploración visual de su estructura.
Prioridad	Alta

Criterios de aceptación	■ El panel ASTViewer renderiza la representación jerárquica del AST con nodos que indican el tipo de dato retornado por el intérprete. ■ La visualización se actualiza automáticamente tras cada ejecución exitosa, sin necesidad de acción adicional por parte del usuario. ■ El panel es responsivo y se adapta al tamaño del contenedor asignado dentro del layout redimensionable. ■ En modo paso a paso, el panel refleja el AST correspondiente al paso de evaluación actualmente visualizado. ■ No se presentan errores de renderizado ni contenido huérfano durante el ciclo de vida de la sesión.
--------------------------------	--

Tabla D.2: RF-02: Visualización gráfica e interactiva del AST

Código	RF-03
Título	Ejecución y evaluación del programa
Descripción	El sistema debe permitir ejecutar el programa del usuario e interpretar su resultado, siguiendo el proceso de evaluación definido por la gramática y el intérprete generado.
Prioridad	Alta
Criterios de aceptación	■ El sistema transmite los archivos del editor y la expresión de entrada a la ruta <code>POST /api/run</code> y recibe el arreglo de pasos de evaluación. ■ El resultado de cada paso es presentado en la consola con nivel de salida correspondiente: verde (<code>output</code>) para valores, rojo (<code>error</code>) para excepciones capturadas, gris (<code>info</code>) para resultados <code>void</code>. ■ Los errores producidos por <code>eopl:error</code> son capturados por el runtime, limpiados de trazas internas de Racket, y presentados con el prefijo <code>×</code> en color rojo. ■ La ejecución puede ser cancelada por el usuario durante el proceso; el servidor aborta el subproceso Racket correspondiente. ■ El tiempo máximo de ejecución del subproceso Racket es de 15 segundos; transcurrido ese tiempo, el sistema reporta el error de tiempo límite en la consola. ■ El sistema soporta un modo de evaluación paso a paso que permite avanzar la ejecución de forma controlada mostrando el estado del AST y del ambiente en cada paso.

Tabla D.3: RF-03: Ejecución y evaluación del programa

Código	RF-04
Título	Representación del ambiente de ejecución
Descripción	El sistema debe representar el ambiente de evaluación como una estructura jerárquica de frames que se actualiza en cada ejecución, permitiendo al estudiante observar la evolución del estado del programa.
Prioridad	Alta
Criterios de aceptación	■ El panel EnvironmentPanel muestra el ambiente como una pila de frames jerárquicos, donde cada frame lista los bindings activos con su nombre, valor serializado y tipo de dato inferido. ■ El ambiente mostrado corresponde al último paso de evaluación completado; en modo paso a paso, se actualiza con cada avance del usuario. ■ La estructura de frames refleja fielmente la cadena de entornos construida durante la evaluación, permitiendo al estudiante identificar el alcance léxico de cada variable. ■ El panel permanece visible y no colapsa ante la ausencia de bindings en el ambiente actual.

Tabla D.4: RF-04: Representación del ambiente de ejecución

Código	RF-05
Título	Biblioteca de ejemplos
Descripción	El sistema debe ofrecer una biblioteca de ejemplos predefinidos orientados a los indicadores de logro identificados como base conceptual del prototipo, con el propósito de facilitar el aprendizaje mediante casos de uso concretos.
Prioridad	Media
Criterios de aceptación	■ Los ejemplos son cargados desde el sistema de archivos del servidor durante el renderizado inicial y transmitidos al cliente como propiedades del componente orquestador. ■ El sistema presenta la lista de ejemplos disponibles en el modal de bienvenida con nombre y descripción breve de cada uno. ■ Al seleccionar un ejemplo, los archivos del editor son reemplazados por los archivos correspondientes sin pérdida de estado de la interfaz. ■ Los ejemplos cubren los cinco indicadores de logro seleccionados en el diagnóstico: IL 1.1.6, IL 1.2.3, IL 1.2.6, IL 1.2.7 e IL 1.3.1. ■ Cada ejemplo es compatible con el pipeline de gramática y puede ser ejecutado sin modificaciones adicionales.

Tabla D.5: RF-05: Biblioteca de ejemplos

Código	RF-06
Título	Soporte y validación de sintaxis en Racket
Descripción	El sistema debe ser capaz de reconocer la sintaxis del lenguaje Racket en el editor y validar la gramática BNF ingresada por el usuario, reportando errores de forma clara y oportuna.
Prioridad	Alta
Criterios de aceptación	■ El editor Monaco provee resaltado de sintaxis para archivos con extensión <code>.rkt</code>. ■ El pipeline de gramática detecta y reporta errores léxicos en la especificación BNF durante el procesamiento; los mensajes son presentados en el modal de gramática antes de generar los archivos. ■ El sistema no genera los archivos del intérprete ni habilita la ejecución cuando la gramática BNF contiene errores léxicos o sintácticos. ■ Los errores de evaluación producidos por el intérprete Racket son presentados en la consola sin exponer trazas internas del runtime al usuario.

Tabla D.6: RF-06: Soporte y validación de sintaxis en Racket

Código	RF-07
Título	Visualización de los componentes internos del intérprete
Descripción	El sistema debe permitir al estudiante visualizar los archivos que componen el intérprete generado, mostrando su estructura con distinción entre las secciones técnicas y las secciones destinadas a la implementación del estudiante.
Prioridad	Media
Criterios de aceptación	■ El panel de edición organiza los archivos del intérprete en pestañas independientes, accesibles mediante navegación por clic. ■ Las líneas generadas automáticamente por el pipeline son marcadas como de solo lectura y presentadas con distinción visual respecto a las líneas editables. ■ Las líneas destinadas a la implementación del estudiante (stubs de <code>eval-program</code>, <code>eval-expression</code>, <code>apply-primitive</code>) permanecen activas y editables. ■ No se permite la modificación de las líneas protegidas durante la sesión de trabajo.

Tabla D.7: RF-07: Visualización de los componentes internos del intérprete

Código	RF-08
Título	Plantilla base editable y exportación del entorno
Descripción	El sistema debe proporcionar una plantilla base del intérprete donde el estudiante pueda completar la implementación, y permitir la descarga del proyecto completo en formato .zip compatible con la librería EOPL.
Prioridad	Alta
Criterios de aceptación	■ Al confirmar la gramática, el pipeline genera automáticamente los cuatro archivos del intérprete con los stubs de las funciones a implementar, cargándolos en el editor sin intervención adicional del usuario. ■ Las secciones de infraestructura técnica del intérprete están protegidas y no son modificables por el usuario. ■ Las secciones de implementación están identificadas con comentarios <code>;; TODO</code> que orientan al estudiante sobre qué debe completar. ■ El sistema genera un archivo .zip con todos los archivos del editor en su estado actual, incluyendo las modificaciones realizadas por el usuario. ■ El archivo descargado incluye una estructura de directorios compatible con la librería EOPL y puede ser abierto directamente en DrRacket.

Tabla D.8: RF-08: Plantilla base editable y exportación del entorno

D.2. Requisitos No Funcionales

Código	RNF-01
Título	Usabilidad
Descripción	El sistema debe ser intuitivo y operable por estudiantes sin experiencia avanzada en compiladores, minimizando la carga cognitiva extrínseca impuesta por la interfaz.
Prioridad	Alta

Criterios de aceptación	▪ Los usuarios pueden iniciar una sesión de trabajo, definir una gramática y ejecutar un programa sin consultar documentación externa. ▪ Los mensajes de error presentados en la consola son comprensibles sin conocimiento del funcionamiento interno de Racket. ▪ La distribución de paneles sigue un orden visual coherente con el flujo de trabajo del estudiante: definición de gramática, edición del intérprete, ejecución y observación de resultados. ▪ Las secciones del editor que no deben ser modificadas están diferenciadas visualmente de forma que el estudiante pueda identificarlas sin instrucción explícita.
--------------------------------	--

Tabla D.9: RNF-01: Usabilidad

Código	RNF-02
Título	Rendimiento
Descripción	El sistema debe completar la generación del AST y la evaluación del programa en un tiempo que no interrumpa el flujo cognitivo del estudiante durante la exploración.
Prioridad	Alta
Criterios de aceptación	▪ La transformación BNF a archivos Racket se completa en el cliente en menos de 1 segundo para gramáticas de hasta 20 producciones. ▪ La respuesta de la ruta <code>/api/run</code> para programas de complejidad habitual en la asignatura FLP se recibe en menos de 5 segundos bajo condiciones normales de red. ▪ El tiempo máximo de ejecución del subproceso Racket está limitado a 15 segundos; transcurrido ese tiempo, el sistema notifica al usuario y libera los recursos asociados. ▪ No se producen bloqueos del hilo principal del navegador durante la ejecución de la petición al servidor.

Tabla D.10: RNF-02: Rendimiento

Código	RNF-03
Título	Accesibilidad web
Descripción	El sistema debe ejecutarse en navegadores web modernos sin requerir instalación de software adicional, plugins ni autenticación por parte del usuario.
Prioridad	Alta

Criterios de aceptación	■ La aplicación carga y opera correctamente en las versiones actuales de Google Chrome, Mozilla Firefox y Microsoft Edge. ■ No se requiere instalación de extensiones, plugins ni entornos de ejecución adicionales en el equipo del usuario. ■ El acceso al sistema no requiere registro ni autenticación de ningún tipo. ■ La interfaz es funcional en resoluciones de pantalla iguales o superiores a 1280×720 píxeles.
--------------------------------	---

Tabla D.11: RNF-03: Accesibilidad web

Código	RNF-04
Título	Mantenibilidad
Descripción	El código fuente debe estar organizado de forma modular, con separación explícita de responsabilidades, para facilitar la incorporación de mejoras futuras.
Prioridad	Media
Criterios de aceptación	■ Las definiciones de tipos e interfaces TypeScript residen exclusivamente en el directorio <code>app/types/</code>, con importaciones explícitas desde los módulos que las consumen. ■ Las funciones puras y utilitarios que no dependen del estado React residen en <code>app/lib/</code> y no en los archivos de componentes. ■ El pipeline de gramática está segmentado en módulos independientes (lexer, parser, generadores) con responsabilidades no solapadas. ■ Es posible añadir un nuevo generador de archivo Racket incorporando un módulo en <code>app/lib/</code> sin modificar los componentes de interfaz.

Tabla D.12: RNF-04: Mantenibilidad

Código	RNF-05
Título	Escalabilidad
Descripción	La arquitectura del sistema debe permitir incorporar nuevos ejemplos, gramáticas de mayor complejidad o características adicionales sin afectar la estabilidad de las funcionalidades existentes.
Prioridad	Media

Criterios de aceptación	■ La incorporación de nuevos ejemplos se realiza añadiendo archivos al directorio de ejemplos del servidor, sin modificar código de componentes ni de la API. ■ El pipeline de gramática procesa gramáticas con hasta 20 producciones sin degradación observable del rendimiento en el cliente. ■ La adición de un nuevo panel de visualización puede realizarse como un componente React independiente integrable en el layout sin alterar el comportamiento de los paneles existentes. ■ El sistema de contenedores Docker permite escalar el servicio de ejecución de forma independiente al servidor de la interfaz.
--------------------------------	--

Tabla D.13: RNF-05: Escalabilidad